TenderoBot — Reporte técnico de avance 3

Oct 31, 2026 · commit `23fbaa8`

Resumen

El bot de Telegram @CO_Tendero_Bot, la página pública, el panel del proveedor y el nuevo panel de administración corren en local sobre un solo servidor FastAPI, conectados a Grok (a través de la API del reto), a PostgreSQL en Neon, a Gmail para los correos y a OpenStreetMap para el mapa. Las **217 pruebas** automáticas pasan (eran 119). Este avance agrega lo necesario para operar con usuarios reales: tenderos y proveedores se registran desde una página pública con el correo verificado, cada solicitud la aprueba quien corresponde, hay un solo login y un panel de administración que maneja a los proveedores. También corrige tres errores del pedido. Sigue faltando el despliegue en Render (fase 6).

Fase	Alcance	Estado
1	Estructura, configuración, modelos, migraciones y seed	Terminada; 6 migraciones nuevas, 16 tablas. El seed exige una cuenta del panel con contraseña fuerte
2	Motor de inventario y pruebas	Terminada; ahora descuenta lo que viene en camino
3	Cliente Grok, matching y esquemas	Sin cambios
4	Bot de Telegram con todos los flujos	Terminada; entrada con el celular verificado, `/recibido` con revisión y botones atados a su borrador
5	Panel web del proveedor	Terminada; solicitudes de tenderos, alta y edición de tiendas, productos bloqueados
6	Docker, despliegue y README	README al día; Docker y despliegue pendientes
+	Página pública, registro, login único y administración (fuera del plan original)	Terminada

Arquitectura y stack

Un solo proceso FastAPI aloja el bot, la página pública, los dos paneles, el motor de inventario y la tarea de fondo. Los cuatro servicios externos son gratuitos. Grok sigue siendo el único que usa IA.

Un servidor y cuatro servicios externos gratuitos; la IA solo extrae datos



El bot guarda ventas, conteos, pedidos y recibos solo después de que el tendero confirma con un botón. Stack: Python 3.12, FastAPI 0.143, python-telegram-bot 22.8, SQLAlchemy 2.1 asíncrono con asyncpg, Alembic 1.20, Pydantic 2.14, httpx, rapidfuzz, Jinja2, Chart.js, bcrypt 5, `smtplib` (biblioteca estándar), Leaflet 1.9.4 y pytest.

Usuarios y acceso

Hay tres papeles y **cada correo tiene uno solo**: un correo de proveedor no puede pedir acceso como tienda, ni al revés, y el servidor lo revisa aunque el formulario se envíe sin pasar por la página. Una solicitud rechazada no cuenta como papel: la persona puede volver a intentar.

Papel	Cómo entra	Quién lo aprueba	Qué ve
Tendero	Bot de Telegram, compartiendo su celular verificado. En la web, un código al correo para ver su solicitud	Su proveedor	Su tienda: ventas, stock, alertas, pedido
Proveedor	`/login` con correo y contraseña; la crea él con un enlace de un solo uso	El administrador	Sus tiendas, productos, pedidos y solicitudes
Administrador	`/login` con correo y contraseña; la cuenta solo se crea por comando	Nadie (comando local)	Todos los proveedores y el resumen del negocio

- **Login único** en `/login`: revisa siempre las dos clases de cuenta, así la respuesta tarda igual y no revela de qué tipo es un correo. Las entradas viejas (`/panel/login` , `/admin/login`) redirigen.
- Contraseñas de 12 caracteres o más. Las de prueba o muy conocidas (por ejemplo la del antiguo usuario demo) se rechazan aunque sean correctas.
- Enlaces de contraseña: un solo uso, 7 días para crearla y 24 horas para cambiarla; en la base solo queda el hash SHA-256. El administrador nunca conoce la contraseña.

- Proveedor y administrador usan claves distintas dentro de la misma cookie firmada: abrir una sesión no cierra la otra. El panel revisa en cada petición que el proveedor siga activo.

Alta de tenderos y proveedores

Toda solicitud empieza igual: la persona escribe su correo y recibe un código de 6 dígitos. El código vence en 10 minutos, admite 5 intentos y solo se guarda su HMAC. Se pide máximo uno por minuto y 5 por hora por correo, y 5 cada 10 minutos por IP. Hay además un campo oculto que solo llenan los robots.

	Tendero	Proveedor
Datos	Tienda, dueño, cédula o NIT, celular, ciudad, barrio, dirección, horario de recibo, forma de pago, ubicación opcional en el mapa	Razón social, nombre comercial, NIT con dígito de verificación, matrícula mercantil, representante legal y cédula, contacto (nombre, cargo, celular), sede, cobertura, categorías, vehículos, tiendas que atiende
Validación	Celular colombiano, horario dentro del reparto, una sola solicitud en revisión por celular y por correo	Dígito de verificación del NIT con el método de la DIAN (probado con NIT reales), NIT único, declaración de RUT y Cámara de Comercio vigentes
Revisión	El proveedor escoge los días de ruta y la anticipación, o rechaza con motivo	El administrador aprueba o rechaza con motivo. Las categorías reguladas (licores y cigarrillos) se marcan para revisar
Al aprobar	Se crea la tienda. Si ya abrió el bot queda vinculada y recibe el aviso por Telegram; si no, por correo	Se crea el proveedor con su cuenta y le llega el enlace para crear su contraseña

Horario de recibo. Hora de inicio y de final escritas en dos campos. Deben quedar entre 6 a. m. y 8 p. m. y no empezar ni terminar en la hora del almuerzo (12 m. a 1 p. m.); un horario que pasa por encima del almuerzo significa "todo ese tiempo menos el almuerzo". Las reglas viven en `app/hours.py` y se cambian en un solo lugar.

Ubicación de la tienda en el mapa

Es opcional y se puede marcar desde cualquier lugar, no solo estando en la tienda. En OpenStreetMap casi ninguna casa de Colombia tiene número, pero las calles y carreras sí. Por eso la página lee la dirección colombiana: `Cl. 23 #4-50` significa *sobre la Calle 23, a unos 50 m de la Carrera 4*.

1. Entiende las abreviaturas Cl, Cll, Cra, Kr, Cr, Av. Calle, Dg y Tv, con o sin "#" o "No.", y con letras (4A, 12B).
2. Pide a Nominatim el trazado de la calle y de la carrera, solo dentro de la ciudad escrita, y calcula dónde se cruzan. Si encuentra la esquina siguiente, avanza los metros de la placa.
3. Si el cruce no está en el mapa, cae a la calle, luego al barrio y luego a la ciudad, y le dice al tendero qué encontró. El pin se arrastra para corregir.
4. Respeta el límite de Nominatim (una consulta por segundo). El mapa (Leaflet) solo se descarga si el tendero lo abre.

Probado con una dirección real de Piedecuesta: la esquina calculada coincide al decimal con la de Overpass. Se descartó Overpass porque en las pruebas respondió con errores 502 o sin respuesta.

Motor de inventario y pedidos

Las reglas de stock, venta diaria, cobertura, stock de seguridad, ABC, riesgo y estancado no cambian. Cambian tres cosas:

Cambio	Regla
Mercancía en camino	Los pedidos enviados y no recibidos (con entrega hasta 2 días atrás) se restan del pedido sugerido. Antes, volver a escribir /pedido sugería lo mismo otra vez
Un pedido por entrega	`/pedido` y el botón Enviar se niegan si ya hay un pedido para esa entrega: "Ya enviaste el pedido para el jueves…"
Recibido con revisión	`/recibido` muestra lo que traía el pedido más antiguo. El tendero corrige lo que llegó ("coca litro 8") y solo eso se suma al inventario; el faltante se informa

Bot de Telegram

Los cambios del bot son sobre la entrada, la confianza en los botones y los avisos:

Entrada	Respuesta del bot
`/start` y Compartir mi número	Telegram manda el número verificado de la cuenta; un contacto reenviado se rechaza. Si el número es de una tienda, entra; si tiene una solicitud en revisión, se guarda el chat para avisarle; si fue rechazada, muestra el motivo
Autorización de datos (Ley 1581)	Se guarda por cuenta de Telegram: si otra cuenta entra con el mismo número, se le pide de nuevo
Botones Confirmar, Cancelar y Enviar	Llevan el id de su borrador y se leen con `SELECT … FOR UPDATE`. Un botón de un mensaje anterior responde "Ese mensaje ya no está vigente"; un doble toque no guarda dos veces
`/salir` y sesión abierta	Cierra la sesión del chat. Tras 24 horas sin uso avisa, de día (9 a. m. a 7 p. m.) y una sola vez, que la sesión sigue abierta
Proveedor suspendido	Responde que el proveedor está suspendido y no registra nada; tampoco envía recordatorios

Paneles web

Los dos paneles usan plantillas Jinja2 con el mismo diseño. Cada ruta del proveedor valida que la tienda, el producto o la solicitud sean suyos (404 si no). Las del administrador exigen su sesión. Las acciones que se pueden deshacer piden motivo, y eliminar un proveedor exige escribir su nombre exacto.

Ruta	Qué hace
`/panel/solicitudes`	Tenderos que piden acceso, con mapa y horario; aprobar (días de ruta y anticipación) o rechazar con motivo
`/panel/tiendas/nueva`, `…/datos`	Crear una tienda a mano y editar sus datos; si cambian los días de ruta, el bot vuelve a preguntar los días de entrega
`/panel/activar/{token}`	Página del enlace del correo para crear o cambiar la contraseña

Ruta	Qué hace
`/admin`	Resumen: proveedores activos y suspendidos, solicitudes, tiendas que usan el bot y valor de los pedidos de 30 días
`/admin/solicitudes`	Solicitudes de proveedores con todos sus datos; aprobar o rechazar con motivo
`/admin/proveedores/{id}`	Datos de la empresa; cuentas (agregar, enviar enlace, quitar); productos (bloquear con motivo o desbloquear); suspender, reactivar o eliminar

Un producto bloqueado sale del catálogo y del bot. El proveedor lo ve como "Bloqueado por TenderoBot" con el motivo y no lo puede reactivar. La página pública y los paneles cargan con animaciones que solo usan transform y opacity; además se quitaron el fondo fijo y el desenfoque de las tarjetas, que se repintaban en cada cuadro del scroll.

Costo por tarea

Las tareas con IA no cambian. Todo lo nuevo usa servicios gratuitos:

Tarea	Servicio	Costo
Venta por texto, foto o cambio al pedido	Grok	1 llamada · ~800 a 1.000 tokens de entrada (estimado, sin medir)
Venta por voz	Grok	2 llamadas (/stt + extracción)
Código de registro, avisos y enlaces	Gmail con contraseña de aplicación	Gratis, dentro del límite diario de Gmail
Buscar la dirección y mostrar el mapa	Nominatim y mapas de OpenStreetMap	Gratis; 3 a 5 consultas por búsqueda, una por segundo
Paneles, stock, alertas, pedido, recordatorios	Ninguno	Sin costo

Pruebas y verificación

Las 217 pruebas de pytest pasan (`python -m pytest -q`). Las de IA usan respuestas simuladas, sin llamar a la API.

Archivo	Pruebas
`tests/test_web.py` : login único, contraseñas, rutas protegidas, plantillas sin variables vacías	30 aprobadas
`tests/test_bot_gate.py` : entrada con el celular, consentimiento, días de entrega, sesión	28 aprobadas
`tests/test_engine.py` : stock, cobertura, ABC, entregas, mercancía en camino	27 aprobadas
`tests/test_orders.py` : cambios al pedido, existencias, recordatorios	25 aprobadas
`tests/test_catalog.py` : formulario y cambios del catálogo	22 aprobadas
`tests/test_onboarding.py` : solicitudes de tenderos, aprobación y avisos	19 aprobadas

Archivo	Pruebas
`tests/test_ai.py` : validación, reintento y matching con Grok simulado	16 aprobadas
`tests/test_hours.py` : horario de recibo, reparto y almuerzo	13 aprobadas
`tests/test_verification.py` : código de correo y un papel por correo	13 aprobadas
`tests/test_admin.py` : NIT, solicitud de proveedor, enlaces, productos bloqueados	12 aprobadas
`tests/test_drafts.py` : botones atados a su borrador y `/recibido`	12 aprobadas

Además se corrieron recorridos completos contra la base real en Neon, con cuentas de prueba borradas al terminar: registro, solicitud y aprobación de un proveedor, creación de su contraseña con el enlace, producto bloqueado que no se puede reactivar, suspensión y reactivación, manejo de cuentas, eliminación, y la regla de un papel por correo. Las páginas se revisaron con capturas en escritorio y en 390 px. Siguen sin cobertura automática el recorrido real en Telegram y la entrega de correos a bandejas reales; se prueban a mano.

Configuración y seguridad

- Los secretos viven solo en `.env` , fuera de git. Variables nuevas: `SMTP_HOST` , `SMTP_PORT` , `SMTP_USER` , `SMTP_PASSWORD` , `SMTP_FROM` , `PANEL_EMAIL` y `PANEL_PASSWORD` (antes `DEMO_PASSWORD`). Ya no hay credenciales publicadas en el README ni en `.env.example` .
- Cuentas por comando: `python -m scripts.admin_user` crea el administrador y `python -m scripts.panel_user` cambia el acceso de un proveedor sin borrar datos; ambos piden la contraseña oculta.
- Lo que escriben tenderos y proveedores se escapa en las páginas y en los correos; las confirmaciones en JavaScript reciben los datos como JSON.
- Los correos van en HTML y en texto plano, con el logo dentro del mensaje; los logs no incluyen destinatarios ni contenidos.
- Se mantienen el límite de 20 mensajes por minuto en el bot, 5 intentos por minuto en el login con cuenta regresiva y los límites de fotos (10 MB) y audios (2 minutos), que no se guardan.

Pendientes

Para la demo sigue faltando el despliegue: hoy todo responde solo mientras el servidor corre en el PC local.

- Dockerfile para Render, modo webhook, `PUBLIC_BASE_URL` real (lo usan los enlaces de los correos) y UptimeRobot contra `/health` .
- Aviso al proveedor cuando un tendero envía un pedido; hoy solo aparece en su panel.
- Varios proveedores por tienda: hoy cada tienda tiene uno, aunque un tendero real le compra a varios.
- Registrar los tokens que reporta Grok en cada llamada para medir el costo real por tarea.
- La ubicación en el mapa llega a la cuadra, no a la puerta, y depende de que OpenStreetMap tenga las calles del barrio.
- Herramientas, ejecuciones y casos de evaluación de las siguientes semanas del reto.